DSC 257R: Unsupervised learning

Homework 8

1. *Experiments with the EM algorithm.* In this problem, we'll generate data from a mixture of Gaussians and then see whether the EM algorithm is able to recover the correct clustering and the correct parameters of the mixture model.

   (a) Consider a mixture of four Gaussians in $\mathbb{R}^2$:

   $$p(x) = \pi_1 N(\mu_1, \Sigma_1) + \pi_2 N(\mu_2, \Sigma_2) + \pi_3 N(\mu_3, \Sigma_3) + \pi_4 N(\mu_4, \Sigma_4)$$

   where the mixing weights are

   $$\pi_1 = 0.1,\ \pi_2 = 0.2,\ \pi_3 = 0.5,\ \pi_4 = 0.2,$$

   the cluster means are

   $$\mu_1 = \begin{bmatrix} 7 \\ 6 \end{bmatrix},\ \mu_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix},\ \mu_3 = \begin{bmatrix} -1 \\ 4 \end{bmatrix},\ \mu_4 = \begin{bmatrix} 5 \\ -2 \end{bmatrix},$$

   and the covariance matrices are

   $$\Sigma_1 = \begin{bmatrix} 1 & 0 \\ 0 & 25 \end{bmatrix},\ \Sigma_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},\ \Sigma_3 = \begin{bmatrix} 9 & 1 \\ 1 & 1 \end{bmatrix},\ \Sigma_4 = \begin{bmatrix} 16 & -6 \\ -6 & 4 \end{bmatrix}.$$

   Write a routine that generates 400 points at random from this mixture model. Plot the points, using colors to distinguish the four components. Put this plot in your solution.

   (b) Write a routine that takes a two-dimensional Gaussian mixture (means and covariance matrices) and plots ellipses corresponding to each component. You might find `matplotlib.patches.Ellipse` helpful for this. In particular, for a Gaussian whose mean is named `mu` and whose covariance is named `sigma`, the following lines will create a suitable ellipse and add it to your figure `ax`.

   ```
   from matplotlib.patches import Ellipse
   eigvals, eigvecs = np.linalg.eigh(sigma)
   direction = eigvecs[0] / np.linalg.norm(eigvecs[0])
   theta = np.arctan2(direction[1], direction[0]) * 180 / np.pi + 180
   stretch = 2.0 * np.sqrt(3.0 * eigvals)
   ellipse = Ellipse(mu, stretch[0], stretch[1], angle=theta, color='red')
   ellipse.set_alpha(0.25)
   ax.add_artist(ellipse)
   ```

   Using this routine, plot all the data points (from (a)) in the same color and then draw four ellipses on the same plot, depicting representative contours of the four Gaussians. Put this plot in your solution.

   (c) Look at the plots from (a) and (b) and check that the shapes of the clusters correspond to what you would expect from the covariance matrices. One of the clusters is small and is partially contained within another cluster. Which clusters are these?

(d) Now run the EM algorithm on the data. For this you can use `sklearn.mixture.GaussianMixture`. You should set `random_state` to something fixed, like 0, and take `init_params` to be `random_from_data`. Plot the data, and using the procedure from (b), draw four ellipses on the same plot, depicting the four Gaussians returned by EM. How close are they to the correct Gaussians?

(e) Plot the data again, but this time color each point according to its **most likely** mixture component. How does this clustering compare to the true component memberships (from (a))? Are the discrepancies understandable?

(f) Finally, let's see how EM's solution evolves over iterations of local search. To do this, pause EM after each iteration and then use the procedure from (b) to draw ellipses for the current Gaussians (along with the data points). Show four of these plots, depicting the progress of EM.

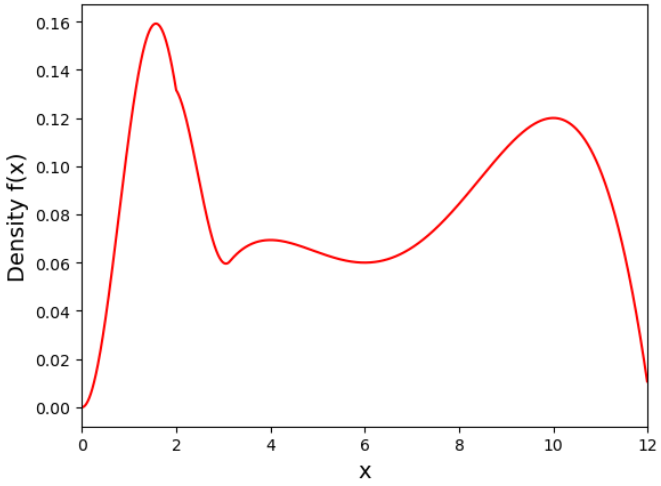
Note: To implement this, you can again use `GaussianMixture`, setting the number of iterations (`max_iter`) appropriately. For instance, you can run it for 1 iteration (from the beginning), then two iterations (with the same initial parameters), then three, and so on.

2. *Kernel density estimation.* Suppose we have samples $x_1, x_2, \ldots, x_n \in \mathbb{R}^d$ from an unknown distribution f, and we want to come up with a model of f. One simple way to do this is using the *kernel density estimator* $\widehat{f}$, which approximates f using a mixture of n Gaussians, each with weight $1/n$, and each centered at one of the data points:

$$\widehat{f}: \ \frac{1}{n}N(x_1, \sigma^2 I_d) + \frac{1}{n}N(x_2, \sigma^2 I_d) + \cdots + \frac{1}{n}N(x_n, \sigma^2 I_d).$$

Here σ must be selected carefully. It is usually called the *bandwidth* parameter.

In this problem, we will apply kernel density estimation to a one-dimensional ($d = 1$) data set. The density f looks like this:



The file `samples.txt` contains 1,000 samples drawn from f (using rejection sampling). Begin by loading in this data.

(a) Plot a histogram of the 1,000 samples using 20 bins. It should (very roughly) resemble the distribution f shown above.

(b) Now apply kernel density estimation using *just the first 100 samples* in `samples.txt`, that is, $n = 100$. You can use `sklearn.neighbors.KernelDensity` for this, making sure to set `kernel='gaussian'`. For the `bandwidth` parameter, try the following settings: 0.25, 0.5, 0.75, 1.0.

- For each of the four bandwidth settings, show a plot of the resulting density $\widehat{f}$.
- What changes as the bandwidth increases? Give a one- or two-sentence description.
- In this case, which of the four bandwidth settings yields a model $\widehat{f}$ that is closest to the original f, in your opinion?

(c) Now show the plots of the kernel density estimate using all 1000 samples in `samples.txt`, that is, $n = 1000$. Use the same four settings of the bandwidth.